OS Orange Problem Unit 4

Name : Kushal Jayavarapu

SRN : PES1UG24CS246

Section : 4D

Phase 1 :

```
gcc -o test_objects test_objects.o object.o -lcrypto
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ ./test_obje
cts
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bb
e9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea0922
7ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
All Phase 1 tests passed.
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ find .pes/o
bjects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda47
1e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e1
1ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
b9aafc12
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$
```

Phase 2 :

```
24CS246-pes-vcs
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$
```

```
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ find .pes/o
bjects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda47
1e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e1
1ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
b9aafc12
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ xxd $(find
.pes/objects -type f | head -1) | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$
```

Phase 3 :

```
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ echo "hello
 world" > file.txt
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ ./pes add f
ile.txt
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ ./pes statu
s
Staged changes:
  staged:     file.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree
  untracked:  a.txt
  untracked:  test_tree.c
  untracked:  test_objects.c
  untracked:  index.h
  untracked:  index.c
  untracked:  test_objects
  untracked:  pes.h
  untracked:  pes.c
  untracked:  object.c
  untracked:  commit.h
  untracked:  tree.h
  untracked:  tree.c
  untracked:  b.txt
  untracked:  Makefile
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  README.md
  untracked:  test_sequence.sh
```

```
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ cat .pes/in
dex
100644 0bd69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143
d 1776778873 12 file.txt
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$
```

Phase 4 :

```
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ ./pes log
commit 10a25496e660c1cdf27edd08c9ee1395f8e1bc6f687a31546f05b4627d7295e
4
Author: PES User <pes@localhost>
Date:   1776780416

    third commit

commit fde3e779ff15eeac590ddf50ca6850b00110a51fa84e2344781c2e1fdfc8604
8
Author: PES User <pes@localhost>
Date:   1776780410

    second commit

commit b6b3408ad06eb6442c0cb32c93aca544d5436786d0ebc247ce8472f1816c4d4
5
Author: PES User <pes@localhost>
Date:   1776780378

    first commitst commit<peQ

kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$
```

```
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ find .pes -
type f | sort
.pes/HEAD
.pes/index
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e
9a23143d
.pes/objects/10/a25496e660c1cdf27edd08c9ee1395f8e1bc6f687a31546f05b462
7d7295e4
.pes/objects/78/8e1460a6b9266b381524999a44168985fa315025fcf6bc8dc9a162
a9226386
.pes/objects/b6/b3408ad06eb6442c0cb32c93aca544d5436786d0ebc247ce8472f1
816c4d45
.pes/objects/fd/e3e779ff15eeac590ddf50ca6850b00110a51fa84e2344781c2e1f
dfc86048
.pes/refs/heads/main
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$
```

```
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ cat .pes/re
fs/heads/main
10a25496e660c1cdf27edd08c9ee1395f8e1bc6f687a31546f05b4627d7295e4
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ cat .pes/HE
AD
ref: refs/heads/main
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$
```

Final Integration :

```
kushal@LenovoSaltzman:~/pes-project/PESXUG24CS246-pes-vcs$ make test-i
ntegration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: fa3f8508658e... Initial commit

Log after first commit:
commit fa3f8508658e6eb3485e5a3aef28a50449c7acf542f85d7e656c115f2b36536
7
Author: PES User <pes@localhost>
Date:   1776780504
```

```
    Initial commit


--- Second Commit ---
Committed: 2157c2f0f665... Update file.txt

--- Third Commit ---
Committed: 37c0972bff1b... Add farewell

--- Full History ---
commit 37c0972bff1bb7a66e154d7b58b1599746497ca39dd07428a0c7a8c0fbf9a7b
7
Author: PES User <pes@localhost>
Date:   1776780504

    Add farewell

commit 2157c2f0f665c16bfdfed72a1117c9c392eb3350297b8d4fecc8a9ae8c64a95
e
Author: PES User <pes@localhost>
Date:   1776780504

    Update file.txt

commit fa3f8508658e6eb3485e5a3aef28a50449c7acf542f85d7e656c115f2b36536
7
Author: PES User <pes@localhost>
Date:   1776780504

    Initial commital commiteQ
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
37c0972bff1bb7a66e154d7b58b1599746497ca39dd07428a0c7a8c0fbf9a7b7

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e
9a23143d
.pes/objects/21/57c2f0f665c16bfdfed72a1117c9c392eb3350297b8d4fecc8a9ae
8c64a95e
.pes/objects/37/c0972bff1bb7a66e154d7b58b1599746497ca39dd07428a0c7a8c0
fbf9a7b7
.pes/objects/4b/856786f9ff858e6686cf3fb6f2db1e6085f074037d2d9ad01df058
c4403345
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d669579
7533f50d
.pes/objects/5e/4124ec19b1fed4d23c7a7b7b0e8fc98b597a569115dc06127796dc
ae4a4e5a
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08
de0ccd92
.pes/objects/d9/94faed5cd1caa49379a8ebd66b076b6d579c26460a5e19a5b93bc8
71ae3641
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b
3b3da6ff
.pes/objects/fa/3f8508658e6eb3485e5a3aef28a50449c7acf542f85d7e656c115f
2b365367

=== All integration tests completed ===
```

Phase 5 & 6: Analysis Answers

Q5.1: Implementing pes checkout <branch>

A branch is stored as a file in .pes/refs/heads/<branch> containing a commit hash.

To implement pes checkout <branch>:

1. Update .pes/HEAD to:
2. ref: refs/heads/<branch>
3. Read the commit hash from .pes/refs/heads/<branch>.
4. Load the commit and extract its tree.

5. Update the working directory:
   - Recreate files from the tree (read blobs and write contents)
   - Remove files that are not present in the target tree
6. Update the index to match the checked-out tree.

Why this is complex:

- Files must be safely overwritten or removed
- Directory structure must be rebuilt
- Local uncommitted changes may cause conflicts
- The operation must avoid leaving the repository in a partial state

Q5.2: Detecting Dirty Working Directory Conflicts

To detect conflicts:

1. For each file in the index:
   - Read the file from the working directory
   - Compute its hash
   - Compare with the hash stored in the index
2. If the hashes differ → the file is modified (dirty)
3. Compare with the target branch's version:
   - If:
   - working file ≠ index AND index ≠ target branch
   - then checkout would overwrite local changes
4. In such a case, refuse checkout.

Q5.3: Detached HEAD

A detached HEAD means .pes/HEAD contains a commit hash directly instead of a branch reference.

What happens:

- New commits are created normally
- However, no branch pointer is updated
- These commits become unreferenced (dangling)

Recovery:

- If the commit hash is known, create a new branch:
- echo <commit-hash> > .pes/refs/heads/recovered
- This restores access to those commits

Q6.1: Garbage Collection Algorithm

Goal: Remove unreachable objects.

Algorithm (Mark-and-Sweep):

1. Create a set reachable.
2. Start from all branch heads and HEAD.
3. Traverse:
   - For each commit → mark it and visit its tree and parent(s)
   - For each tree → mark it and visit blobs/subtrees
4. After traversal:
   - Delete any object in .pes/objects/ not in reachable

Data Structure:

- Use a hash set for efficient lookup

Estimate:

- For 100,000 commits and 50 branches:
  - ~100k commits
  - ~100k trees
  - plus blobs
- Total objects visited ≈ 200k–500k

Q6.2: GC Race Condition

Running garbage collection concurrently with commit is dangerous.

Race condition:

1. GC scans and identifies reachable objects
2. A commit is in progress and creates new objects
3. These new objects are not yet referenced
4. GC deletes them as unreachable
5. Commit finishes and references deleted objects

Result:

- Repository corruption (missing objects)

How Git avoids this:

- Atomic object writes (temp file → rename)
- Reflogs to retain recent references
- Grace period before deletion
- Locking of references
- Two-phase garbage collection

Conclusion

These mechanisms illustrate that version control systems rely on:

- Graph traversal
- Hash-based integrity
- Careful state management
- Atomic operations